Assignment: 1

Title: Analyzing Sales Data from Multiple File Formats

---

1. Objectives

1. To import and consolidate sales data from multiple file formats (CSV, Excel, JSON, etc.).
2. To preprocess and clean the data for consistency and accuracy.
3. To analyze sales trends, revenue patterns, and product performance.
4. To visualize insights using graphs and charts for decision-making.

---

2. Problem Statement

In real-world businesses, sales data is often stored across heterogeneous file formats such as CSV, Excel, and JSON. Manually consolidating and analyzing such data is time-consuming, error-prone, and inefficient. Therefore, a system is required to automatically extract, integrate, and analyze sales data from multiple file formats, providing meaningful insights for business decisions.

---

3. Outcomes

- Ability to handle heterogeneous file formats programmatically.
- Processed dataset ready for analysis after cleaning and transformation.
- Computed metrics such as total revenue, product-wise sales, and customer trends.
- Visualizations (bar charts, line graphs, pie charts) for better interpretation.
- Enhanced decision-making through automated sales analysis.

---

4. Software and Hardware Requirements

4.1 Software Requirements

- Python (Anaconda/Google Colab/Jupyter Notebook)
- Libraries: Pandas, NumPy, Matplotlib/Seaborn
- Operating System: Windows/Linux/macOS

4.2 Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 500 MB free space
- Display: Standard 1024×768 resolution

---

5. Theory – Concept in Brief

Data analysis involves **data collection, preprocessing, integration, analysis, and visualization**.
Sales data often exists in multiple file formats due to varied sources like **point-of-sale systems, e-commerce platforms, or ERP tools**.

Using **Python Pandas**, multiple file formats can be read into **DataFrames**, merged, and processed. Exploratory Data Analysis (EDA) is then applied to extract **sales trends, seasonality, and outliers**.

---

6. Algorithm

1. Start
2. Import necessary libraries (Pandas, NumPy, Matplotlib).
3. Load sales data from CSV, Excel, and JSON files.
4. Merge the datasets into a single DataFrame.

5. Handle missing values, duplicates, and inconsistencies.
6. Perform calculations (total sales, average revenue, top-selling products).
7. Generate visualizations (line charts, bar graphs, pie charts).
8. Display analysis results.
9. End

---

7. Flowchart

8. Test Cases

Test Case ID	Input	Expected Output	Result
TC1	Load CSV file with sales data	Data loaded successfully	Pass
TC2	Load Excel file with multiple sheets	Correct sheet read into DataFrame	Pass
TC3	Merge CSV + Excel + JSON	Consolidated single dataset	Pass
TC4	Handle missing values	Nulls replaced/dropped appropriately	Pass
TC5	Compute total sales per product	Accurate sales summary	Pass

9. Test Data Set (Example)

CSV File:

Product	Quantity	Price	Date
A	10	100	2024-01-01
B	5	200	2024-01-02

Excel File:

Product	Quantity	Price	Date
C	8	150	2024-01-01
A	12	100	2024-01-03

JSON File:

```
[
  {"Product": "B", "Quantity": 3, "Price": 200, "Date": "2024-01-04"},
  {"Product": "C", "Quantity": 6, "Price": 150, "Date": "2024-01-05"}
]
```

---

10. Mathematical Model

Let:

- Q_i = Quantity sold of product i
- P_i = Price per unit of product i
- n = Total number of sales transactions

Total Sales (TS):

$$TS = \sum_{i=1}^{n} (Q_i \times P_i)$$

Average Sales (AS):

$$AS = \frac{TS}{n}$$

Top Product (TP):

$$TP = \max(Q_i \times P_i)$$

---

11. Conclusion

The project demonstrates the ability to **analyze sales data from multiple file formats** effectively. By automating data integration, cleaning, and visualization, businesses can make data-driven decisions quickly. The framework provides flexibility, scalability, and adaptability to diverse data sources, making it highly applicable in retail and e-commerce industries.

---

12. MCQs

Q1. Which Python library is primarily used for handling CSV, Excel, and JSON sales data?

- a) NumPy
- b) Pandas
- c) Matplotlib
- d) TensorFlow

Q2. Which of the following is NOT a file format commonly used for sales data?

- a) CSV
- b) Excel
- c) JSON
- d) JPEG

Q3. The process of filling or removing missing values in a dataset is known as:

- a) Aggregation
- b) Cleaning
- c) Normalization
- d) Mining

Q4. Which formula calculates total sales in mathematical terms?

- a) $TS=\sum(Qi+Pi)$
- b) $TS=\sum(Qi\times Pi)$
- c) $TS=\sum(Qi/Pi)$
- d) $TS=\sum(Pi-Qi)$

Q5. Which chart is best suited for analyzing product-wise sales contribution?

- a) Line Chart
- b) Scatter Plot
- c) Pie Chart
- d) Histogram

Assignment: 2

Title: Analyzing Weather Data from OpenWeatherMap API

---

1. Objectives

1. To fetch real-time weather data using the OpenWeatherMap API.
2. To preprocess and organize the retrieved weather data.
3. To analyze temperature, humidity, wind speed, and other weather parameters.
4. To visualize weather trends and patterns using charts and graphs.

---

2. Problem Statement

Weather plays a crucial role in decision-making across industries such as agriculture, aviation, logistics, and disaster management. However, raw weather data obtained from APIs is unstructured and requires preprocessing. Analyzing this data in real-time enables forecasting, anomaly detection, and better planning. This project aims to fetch, process, and analyze weather data from the OpenWeatherMap API, providing meaningful insights.

---

3. Outcomes

- Successful retrieval of weather data using API calls.
- Preprocessed and structured dataset ready for analysis.
- Extracted insights on temperature variations, humidity patterns, and wind flow.
- Data visualizations for trend analysis and reporting.
- Improved decision-making in weather-dependent sectors.

4. Software and Hardware Requirements

4.1 Software Requirements

- Python (Anaconda/Google Colab/Jupyter Notebook)
- Libraries: Requests, Pandas, NumPy, Matplotlib/Seaborn
- API Key from OpenWeatherMap
- Operating System: Windows/Linux/macOS

4.2 Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 500 MB free space
- Internet Connection: Required for API calls

---

5. Theory – Concept in Brief

An **API (Application Programming Interface)** allows applications to communicate and exchange data. OpenWeatherMap provides a free and paid API to access **real-time and historical weather data**.

Using Python's **requests library**, JSON responses from the API can be fetched, parsed, and converted into DataFrames using **Pandas**.

Key weather parameters include **temperature, pressure, humidity, wind speed, and cloud cover**. Analyzing this data reveals weather patterns, seasonal changes, and anomalies.

---

6. Algorithm

1. Start
2. Import libraries (Requests, Pandas, Matplotlib).
3. Obtain API key from OpenWeatherMap.

4. Send GET request to API with city/location parameters.
5. Receive JSON response containing weather data.
6. Parse JSON and extract relevant fields (temperature, humidity, wind, etc.).
7. Convert data into Pandas DataFrame.
8. Perform analysis (average temperature, max/min humidity, wind trends).
9. Generate visualizations (line graphs, bar charts, scatter plots).
10. Display results.
11. End

---

7. Flowchart

8. Test Cases

Test Case ID	Input	Expected Output	Result
TC1	Valid city name ("London")	Weather data retrieved	Pass
TC2	Invalid city name ("XYZ")	Error/No data found	Pass
TC3	Missing API key	Authentication error	Pass
TC4	Parse JSON response	Correct DataFrame created	Pass
TC5	Compute average temperature	Accurate calculation	Pass

9. Test Data Set (Example API Response)

Sample JSON (Simplified):

```
{
  "main": {
    "temp": 298.15,
    "humidity": 82,
    "pressure": 1012
  },
  "wind": {
    "speed": 3.6
  },
  "weather": [
    {"description": "light rain"}
  ]
}
```

Converted DataFrame:

Temperature (K)	Humidity (%)	Pressure (hPa)	Wind Speed (m/s)	Description
298.15	82	1012	3.6	light rain

10. Mathematical Model

Let:
- T = Temperature
- H = Humidity
- W = Wind speed
- P = Pressure
- n = Number of records

Average Temperature (AT):
$AT = (\Sigma T_i) / n$

Average Humidity (AH):
$AH = (\Sigma H_i) / n$

Correlation between Temperature & Humidity (r):
$r = cov(T, H) / (\sigma T \cdot \sigma H)$

---

11. Conclusion

This project demonstrates the use of the **OpenWeatherMap API** for analyzing weather data. It shows how raw JSON responses can be transformed into structured datasets for further analysis. The analysis enables identification of weather patterns, forecasting trends, and supporting industries reliant on climate conditions.

---

12. MCQs

Q1. Which Python library is commonly used to fetch data from APIs?

- a) NumPy
- b) Requests
- c) Matplotlib
- d) TensorFlow

Q2. OpenWeatherMap API returns data in which format?

- a) XML
- b) JSON
- c) CSV
- d) TXT

Q3. Which of the following is NOT provided by the OpenWeatherMap API?

- a) Temperature
- b) Humidity
- c) Wind speed
- d) Student grades

Q4. What is required to access the OpenWeatherMap API?

- a) Database connection
- b) API key
- c) HTML file
- d) CSV file

Q5. Which visualization is most suitable for showing weather trends over time?

- a) Pie Chart
- b) Line Chart
- c) Scatter Plot
- d) Histogram

Assignment: 3

Title: Analyzing Customer Churn in a Telecommunications Company

---

1. Objectives

- To collect and preprocess telecom customer data for churn prediction.
- To identify key factors influencing customer churn.
- To build predictive models for customer churn analysis.
- To visualize churn patterns for strategic decision-making.

---

2. Problem Statement

Customer churn is a critical challenge in the telecommunications industry, where companies lose customers to competitors due to factors such as pricing, service quality, or customer dissatisfaction. Manually identifying churn risk is inefficient. Therefore, a data-driven approach is required to analyze customer behavior, predict churn likelihood, and design retention strategies effectively.

---

3. Outcomes

- Cleaned and preprocessed telecom customer dataset.
- Identification of significant churn indicators such as contract type, tenure, and monthly charges.
- Development of predictive models (Logistic Regression, Decision Trees, Random Forest).
- Visualization of churn distribution and customer segments.
- Strategic insights for reducing churn and improving customer retention.

4. Software and Hardware Requirements

4.1 Software Requirements

- Python (Anaconda / Google Colab / Jupyter Notebook)
- Libraries: Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn
- Operating System: Windows / Linux / macOS

4.2 Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 500 MB free space
- Display: Standard 1024x768 resolution

---

5. Theory – Concept in Brief

Customer churn analysis is the process of identifying customers who are likely to discontinue a service. It involves exploratory data analysis (EDA), feature engineering, and predictive modeling to detect churn risk. Machine learning models help in understanding churn patterns and predicting customer behavior, enabling proactive retention strategies.

---

6. Algorithm

1. Start
2. Import necessary libraries (Pandas, NumPy, Scikit-learn, Matplotlib).
3. Load the telecom customer dataset.
4. Preprocess data (handle missing values, categorical encoding, normalization).
5. Split the dataset into training and testing sets.
6. Train churn prediction models (Logistic Regression, Decision Trees, Random Forest).
7. Evaluate models using accuracy, precision, recall, and F1-score.
8. Visualize churn patterns and important features.
9. End

---

7. Flowchart

8. Test Cases

Test Case ID	Description	Expected Result	Status
TC1	Load dataset with valid file path	Dataset loaded successfully	Pass
TC2	Handle missing values in dataset	Missing values replaced/removed	Pass
TC3	Train model with sufficient data	Model trained without errors	Pass
TC4	Predict churn for test data	Churn predictions generated	Pass
TC5	Evaluate model performance	Performance metrics displayed	Pass

9. Test Data Set (if applicable)

Sample attributes from the telecom churn dataset:

- CustomerID
- Gender
- SeniorCitizen
- Partner
- Dependents
- Tenure
- PhoneService
- InternetService
- Contract
- MonthlyCharges
- TotalCharges
- Churn (Yes/No)

---

10. Mathematical Model (if applicable)

Churn prediction can be represented as a binary classification problem:

Let **X = {x_1, x_2, …, xn}** be the feature set (e.g., tenure, monthly charges).
Let **Y = {0,1}**, where **1 = churn** and **0 = non-churn**.

Logistic Regression Model:

$$P(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta n x n)}}$$

---

11. Conclusion

This project provides a structured approach to analyzing and predicting customer churn in telecommunications. By identifying high-risk customers through predictive models, companies can take proactive retention measures. The approach enhances decision-making, reduces churn, and contributes to long-term customer satisfaction and profitability.

---

12. MCQs with Answers

1. **Which of the following is the main goal of churn analysis?**
    - A. Increase revenue
    - B. Predict customer loss
    - C. Improve product design
    - D. Manage inventory

2. **Which algorithm is commonly used for churn prediction?**
    - A. Logistic Regression
    - B. K-means Clustering
    - C. Apriori Algorithm
    - D. DBSCAN

3. **What type of problem is churn prediction?**
    - A. Regression
    - B. Classification
    - C. Clustering
    - D. Association

4. **Which feature is most likely to affect churn in telecom?**
    - A. Customer tenure
    - B. Invoice number
    - C. File format
    - D. Operating system

5. **What metric is most important in churn prediction?**
    - A. Accuracy
    - B. Precision
    - C. Recall
    - D. F1-score

Assignment: 4

Title: Data Wrangling on Real Estate Market

---

1. Objectives

- To collect real estate data from multiple sources (CSV, Excel, API, Web scraping).
- To clean and preprocess the raw data for analysis.
- To identify and handle missing, duplicate, or inconsistent data.
- To prepare a structured dataset suitable for analytics and visualization.

---

2. Problem Statement

Real estate data is often scattered across multiple formats and sources, making analysis difficult. Raw data may contain missing values, duplicates, inconsistent formats, and irrelevant columns. Effective data wrangling is essential to transform this unstructured data into a clean and usable dataset for decision-making in property valuation, market trends, and investment strategies.

---

3. Outcomes

- Successfully collected real estate data from diverse sources.
- Cleaned, standardized, and structured dataset ready for analysis.
- Removed duplicates, handled missing values, and corrected inconsistent entries.
- Extracted insights such as average property price per area, property type distributions, and market trends.
- Created a dataset suitable for predictive modeling or reporting.

4. Software and Hardware Requirements

4.1 Software Requirements

- Python (Anaconda / Google Colab / Jupyter Notebook)
- Libraries: Pandas, NumPy, Matplotlib, Seaborn, OpenPyXL
- Operating System: Windows / Linux / macOS

4.2 Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 500 MB free space
- Internet Connection: Required for API/Web scraping data

---

5. Theory

Data wrangling (or data munging) is the process of cleaning, structuring, and enriching raw data into a desired format for analysis. In the real estate domain, it involves merging data from multiple sources, correcting inconsistent entries, handling missing values, and standardizing column formats. Proper data wrangling ensures high-quality data for analytics, visualization, and predictive modeling.

---

6. Algorithm

1. Start
2. Import necessary libraries (Pandas, NumPy, Matplotlib, Seaborn).
3. Load real estate datasets from CSV, Excel, or API sources.
4. Inspect data for missing values, duplicates, and inconsistencies.
5. Handle missing data using imputation or removal techniques.

6. Remove duplicate rows and correct inconsistent formatting.
7. Standardize units, column names, and data types.
8. Merge or concatenate datasets if multiple sources are used.
9. Create a clean DataFrame ready for analysis.
10. End

---

7. Flowchart

8. Test Cases

Test Case ID	Description	Expected Result	Status
TC1	Load dataset from CSV/Excel	Dataset loaded successfully	Pass
TC2	Identify missing values	Missing values flagged	Pass
TC3	Remove duplicate entries	Duplicates removed	Pass
TC4	Standardize column names and units	Columns standardized	Pass
TC5	Merge multiple datasets	Single clean DataFrame created	Pass

9. Test Data Set (if applicable)

Sample columns from a real estate dataset:

- PropertyID
- Location
- PropertyType
- Area (sq.ft)
- Bedrooms
- Bathrooms
- Price
- YearBuilt
- ListingDate
- AgentName

---

10. Mathematical Model (if applicable)

Data wrangling does not involve predictive modeling but can be represented as a **data transformation function**:

Let **D_raw** = raw dataset, **D_clean** = cleaned dataset

$$D_{clean}=f(D_{raw})$$

where **f** = data wrangling function including:

- Handling missing values
- Removing duplicates
- Correcting inconsistencies
- Standardizing formats
- Merging multiple datasets

---

11. Conclusion

This project demonstrates how to transform raw, unstructured real estate data into a clean and structured dataset suitable for analysis. Proper data wrangling improves data quality, ensures consistency, and enables meaningful analytics for market trends, property valuations, and informed investment decisions.

---

12. MCQs with Answers

1. **What is the main purpose of data wrangling?**
    - A. Data visualization
    - B. Data cleaning and structuring
    - C. Predictive modeling
    - D. Database design

2. **Which Python library is most commonly used for data wrangling?**
    - A. Matplotlib
    - B. Pandas
    - C. TensorFlow
    - D. OpenCV

3. **Which of the following is a common step in data wrangling?**
    - A. Handling missing values
    - B. Building neural networks
    - C. Designing APIs
    - D. Writing SQL queries

4. **Why is data wrangling important in real estate analysis?**
    - A. To generate fake data
    - B. To improve data quality and consistency
    - C. To increase dataset size only
    - D. To visualize graphs directly

5. **Which action is NOT part of data wrangling?**
    - A. Removing duplicates
    - B. Standardizing formats
    - C. Imputing missing values
    - D. Training a regression model

Assignment: 5

Title: Analyzing Air Quality Index (AQI) Trends in a City

---

1. Objectives

- To collect real-time and historical AQI data from official sources or APIs (e.g., OpenAQ, Government portals).
- To preprocess and clean AQI datasets for analysis.
- To analyze trends in pollutants such as PM2.5, PM10, NO2, CO, and O3 over time.
- To visualize AQI patterns for different seasons, locations, and hours of the day.

---

2. Problem Statement

Air pollution is a significant health and environmental concern in urban areas. Understanding AQI trends is essential for policy-making, public health awareness, and pollution control strategies. Raw AQI data is often inconsistent, missing entries, or scattered across multiple sources, making analysis challenging. This project aims to clean, preprocess, and analyze AQI data to reveal temporal and spatial trends, identify pollution hotspots, and support decision-making.

---

3. Outcomes

- Cleaned and structured AQI dataset for the selected city.
- Identification of pollution patterns over time and across regions.
- Analysis of peak pollution hours and seasonal variations.
- Visualizations such as line graphs, heatmaps, and bar charts to represent AQI trends.
- Insights to support environmental monitoring and policy recommendations.

4. Software and Hardware Requirements

4.1 Software Requirements

- Python (Anaconda / Google Colab / Jupyter Notebook)
- Libraries: Pandas, NumPy, Matplotlib, Seaborn, Requests
- API access (OpenAQ or government data portals)
- Operating System: Windows / Linux / macOS

4.2 Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 500 MB free space
- Internet Connection: Required for API/data download

---

5. Theory – Concept in Brief

Air Quality Index (AQI) quantifies the concentration of pollutants in the air and indicates health risk levels. Key pollutants include PM2.5, PM10, NO2, CO, SO2, and O3. AQI data can be collected hourly, daily, or monthly and usually requires preprocessing to handle missing values, inconsistent formats, and duplicates. Data analysis and visualization help identify pollution trends, seasonal variations, and critical periods of poor air quality.

---

6. Algorithm

1. Start
2. Import necessary libraries (Pandas, NumPy, Matplotlib, Seaborn, Requests).
3. Collect AQI data via APIs or CSV files.
4. Inspect the dataset for missing values, duplicates, or inconsistencies.

5. Handle missing or inconsistent entries (imputation or removal).
6. Convert date/time columns to appropriate datetime formats.
7. Aggregate data by day, month, or year as required.
8. Analyze trends for pollutants (PM2.5, PM10, NO2, CO, O3).
9. Generate visualizations: line charts, heatmaps, bar charts.
10. Draw insights from trends and patterns.
11. End

---

7. Flowchart

8. Test Cases

Test Case ID	Description	Expected Result	Status
TC1	Load AQI dataset from CSV/API	Dataset loaded successfully	Pass
TC2	Identify missing values	Missing values flagged	Pass
TC3	Handle missing and inconsistent entries	Cleaned dataset	Pass
TC4	Convert date/time columns	Datetime format correct	Pass
TC5	Analyze PM2.5 trends	Correct trend visualization	Pass

9. Test Data Set (if applicable)

Sample columns from AQI dataset:

- Date
- Time
- PM2.5
- PM10
- NO2
- CO
- O3
- SO2
- AQI Value
- AQI Category
- Location/Station

---

10. Mathematical Model (if applicable)

AQI computation is usually based on the maximum sub-index of individual pollutants:

$$AQI = \max(I_{PM2.5}, I_{PM10}, I_{NO2}, I_{CO}, I_{O3}, I_{SO2})$$

Where I_x is the sub-index for pollutant x, calculated as:

$$I_x = \frac{(I_{HI} - I_{LO})}{(BP_{HI} - BP_{LO})} \times (C - BP_{LO}) + I_{LO}$$

- C = measured concentration of pollutant
- BP_{HI}, BP_{LO} = breakpoint values for the pollutant
- I_{HI}, I_{LO} = AQI corresponding to breakpoint

---

11. Conclusion

This project demonstrates how to clean, preprocess, and analyze AQI data to reveal temporal and spatial pollution trends. Visualizations and statistical analysis enable identification of critical pollution periods and hotspots, supporting environmental policies and public awareness initiatives. Accurate AQI trend analysis is vital for effective air quality management and urban planning.

---

12. MCQs with Answers

1. **What does AQI stand for?**
    - A. Air Quality Indicator
    - B. Air Quality Index
    - C. Atmospheric Quality Index
    - D. Air Quotient Index

2. **Which pollutant is commonly used in AQI analysis?**
    - A. PM2.5
    - B. Lead
    - C. Ozone layer thickness
    - D. Noise level

3. **Which Python library is used to visualize AQI trends?**
    - A. Seaborn
    - B. TensorFlow
    - C. Requests
    - D. OpenCV

4. **AQI trends are useful for:**
    - A. Traffic management only
    - B. Environmental monitoring and public health
    - C. Predicting stock market trends
    - D. Software development

5. **What is a common step in preprocessing AQI data?**
    - A. Handling missing values
    - B. Running deep learning models
    - C. Data encryption
    - D. Building mobile apps

Assignment: 6

Title: Analyzing Sales Performance by Region in a Retail Company

---

1. Objectives

- To collect and consolidate sales data across different regions from multiple file formats (CSV, Excel, Database).
- To preprocess and clean the data for analysis.
- To analyze sales trends, identify high-performing and low-performing regions.
- To visualize regional sales performance for management insights.

---

2. Problem Statement

Retail companies operate across multiple regions, and sales data is often stored in different formats and sources. Without proper consolidation and analysis, it is difficult to evaluate regional performance, identify growth opportunities, or take corrective actions. This project aims to preprocess and analyze sales data by region to provide insights for strategic decision-making and business growth.

---

3. Outcomes

- Consolidated and cleaned dataset of sales across regions.
- Identification of top-performing and underperforming regions.
- Insights into seasonal sales trends and product-wise performance by region.
- Data visualizations such as bar charts, line graphs, and heatmaps to represent performance.
- Actionable insights to improve sales strategy and regional resource allocation.

4. Software and Hardware Requirements

4.1 Software Requirements

- Python (Anaconda / Google Colab / Jupyter Notebook)
- Libraries: Pandas, NumPy, Matplotlib, Seaborn
- Database connector (if required, e.g., MySQL, SQL Server)
- Operating System: Windows / Linux / macOS

4.2 Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 500 MB free space
- Internet Connection: Required if using online datasets or APIs

---

5. Theory – Concept in Brief

Sales performance analysis involves evaluating the revenue generated across different regions to identify trends, high-performing areas, and opportunities for improvement. Data preprocessing is necessary to clean inconsistent entries, merge multiple data sources, and create a structured dataset. Analytical techniques and visualizations help in interpreting the data for business decision-making and performance optimization.

---

6. Algorithm

1. Start
2. Import necessary libraries (Pandas, NumPy, Matplotlib, Seaborn).
3. Load sales datasets from CSV, Excel, or database.
4. Inspect data for missing values, duplicates, and inconsistencies.

5. Handle missing values and remove duplicate records.
6. Standardize column names, data types, and units.
7. Aggregate sales data by region and time period.
8. Analyze trends such as total sales, growth rate, and product performance.
9. Generate visualizations (bar charts, line charts, heatmaps).
10. Interpret results and identify high and low-performing regions.
11. End

---

7. Flowchart

8. Test Cases

Test Case ID	Description	Expected Result	Status
TC1	Load sales dataset from CSV/Excel	Dataset loaded successfully	Pass
TC2	Handle missing and inconsistent values	Cleaned dataset	Pass
TC3	Remove duplicate entries	Duplicate rows removed	Pass
TC4	Aggregate sales by region	Correct regional totals calculated	Pass
TC5	Generate visualizations	Graphs/plots correctly displayed	Pass

---

9. Test Data Set (if applicable)

Sample columns from a retail sales dataset:

- SalesID
- Region
- StoreID
- ProductID
- ProductCategory
- QuantitySold
- UnitPrice
- TotalSales
- SaleDate

---

10. Mathematical Model (if applicable)

Sales analysis can be represented as an aggregation problem:

Let **S** = dataset with columns {Region, SaleAmount, Date, …}

Total Sales per Region:

$$TS_r = \sum_{i=1}^{n_r} SaleAmount_i$$

Where n_r = number of transactions in region r.

Average Sales per Region:

$$AS_r = \frac{TS_r}{n_r}$$

Growth Rate:

$$GR_r = \frac{TS_{current} - TS_{previous}}{TS_{previous}} \times 100$$

---

11. Conclusion

This project provides a framework to consolidate, clean, and analyze sales data by region in a retail company. By identifying regional performance trends and visualizing sales data, management can make informed decisions, allocate resources effectively, and develop targeted strategies to improve overall sales performance.

---

12. MCQs with Answers

1. **What is the primary goal of sales performance analysis by region?**

    - A. Product design improvement
    - B. Identify high and low-performing regions
    - C. Employee performance review
    - D. Customer segmentation

2. **Which Python library is used for data visualization?**
    - A. NumPy
    - B. Matplotlib
    - C. SQLAlchemy
    - D. Requests

3. **Which action is part of data preprocessing?**
    - A. Handling missing values
    - B. Training neural networks
    - C. Writing mobile apps
    - D. Designing APIs

4. **Total sales per region can be calculated by:**
    - A. Counting products
    - B. Summing SaleAmount column grouped by region
    - C. Averaging quantity sold only
    - D. Sorting dataset alphabetically

5. **Visualizations in sales analysis help in:**
    - A. Cleaning data
    - B. Identifying trends and performance patterns
    - C. Encrypting dataset
    - D. Predicting weather

Assignment: 7

Title: Analysis and Visualization of Stock Market Data

---

1. Objectives

- To collect historical and real-time stock market data from APIs or CSV files.
- To preprocess and clean the stock data for analysis.
- To analyze trends, patterns, and volatility in stock prices.
- To visualize stock performance using charts and graphs for decision-making.

---

2. Problem Statement

Stock market data is complex and voluminous, often including daily price movements, trading volumes, and multiple stock attributes. Analysts and investors need structured data and visualizations to understand market trends, identify profitable opportunities, and manage risks. This project aims to preprocess, analyze, and visualize stock data to derive actionable insights and support investment decisions.

---

3. Outcomes

- Cleaned and structured stock market dataset for analysis.
- Identification of trends, moving averages, and stock price fluctuations.
- Visualizations of stock prices over time (line charts, candlestick charts).
- Insights into high and low performing stocks, volatility patterns, and trading volumes.
- Prepared dataset ready for further financial modeling or forecasting.

4. Software and Hardware Requirements

4.1 Software Requirements

- Python (Anaconda / Google Colab / Jupyter Notebook)
- Libraries: Pandas, NumPy, Matplotlib, Seaborn, yfinance, Plotly
- Operating System: Windows / Linux / macOS

4.2 Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 500 MB free space
- Internet Connection: Required for API or online data sources

---

5. Theory – Concept in Brief

Stock market analysis involves examining historical stock prices, volumes, and related financial indicators to understand trends and make informed decisions. Data preprocessing ensures missing or inconsistent entries are corrected. Visualization tools such as line charts, candlestick charts, and moving averages allow investors to observe patterns, volatility, and performance of stocks over time.

---

6. Algorithm

1. Start
2. Import necessary libraries (Pandas, NumPy, Matplotlib, Seaborn, yfinance, Plotly).
3. Collect stock data via APIs (e.g., Yahoo Finance) or CSV files.
4. Inspect the dataset for missing values, duplicates, or inconsistencies.
5. Handle missing values and standardize column names and data types.
6. Compute additional metrics (e.g., moving averages, returns, volatility).

7. Visualize stock price trends using line charts, candlestick charts, and bar charts.
8. Analyze patterns such as price trends, peaks, and volatility.
9. Draw insights to support investment decisions.
10. End

---

7. Flowchart

8. Test Cases

Test Case ID	Description	Expected Result	Status
TC1	Load stock data from API/CSV	Dataset loaded successfully	Pass
TC2	Handle missing values	Cleaned dataset	Pass
TC3	Remove duplicate entries	Duplicate rows removed	Pass
TC4	Calculate moving averages and returns	Metrics correctly computed	Pass
TC5	Generate visualizations	Charts and plots displayed	Pass

9. Test Data Set (if applicable)

Sample columns from a stock market dataset:

- Date
- Open
- High
- Low
- Close
- Adjusted Close
- Volume
- Ticker Symbol

---

10. Mathematical Model (if applicable)

Stock price analysis can include computation of returns and moving averages:

Daily Return:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} \times 100$$

Where P_t = closing price at day t, P_{t-1} = closing price at previous day.

Moving Average (MA) over n days:

$$MA_t = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$$

Where P_{t-i} = closing price of day $t - i$.

---

11. Conclusion

This project demonstrates how to collect, clean, analyze, and visualize stock market data. By examining trends, volatility, and performance, investors and analysts can make informed decisions. Data preprocessing ensures accuracy, and visualization helps in identifying patterns, price fluctuations, and potential investment opportunities.

---

12. MCQs with Answers

1. **Which library is commonly used to fetch stock market data in Python?**
    - A. Pandas
    - B. yfinance
    - C. Matplotlib
    - D. Seaborn

2. **What is the purpose of calculating moving averages?**
    - A. To predict volume only
    - B. To smooth out short-term fluctuations and highlight trends
    - C. To clean missing values
    - D. To encrypt financial data

3. **Which chart is commonly used to visualize daily stock prices?**
    - A. Pie chart
    - B. Candlestick chart
    - C. Histogram
    - D. Scatter plot

4. **Daily stock return is calculated using which formula?**
    - A. $R_t = P_t + P_{t-1}$
    - B. $R_t = \frac{P_t - P_{t-1}}{P_{t-1}} \times 100$
    - C. $R_t = P_t \times P_{t-1}$
    - D. $R_t = P_t / P_{t+1}$

5. **Why is data preprocessing important in stock market analysis?**
    - A. To remove missing or inconsistent data for accurate analysis
    - B. To generate predictions automatically
    - C. To create 3D visualizations
    - D. To encrypt dataset for security